MULTI-AGENT ORCHESTRATION

Architecture, execution semantics, control boundaries and source-level evidence

AUTHORIAL POSITION | I document the implementation that exists in the Aegis Agent repository, distinguish observed code paths from product-wide guarantees, and do not present unexecuted tests as passing evidence.

01 / System intent

I designed Aegis Agent around six named specialist roles and an explicit coordination layer. The implementation contains both a task-to-specialist dispatch path and a six-specialist response-collection path. These paths have different execution semantics, and I treat them separately rather than describing the entire system as one autonomous swarm.

02 / The six specialist roles

Specialist	Registered responsibility
Alina	Strategic coordination / planning and routing
Kian	Operational execution / monitoring and optimization
Bita	Analysis / synthesis and reporting
Aylin	Quality / validation and review
Ahmad	Security / oversight and governance
Amin	Finance / executive bridge

03 / What the registry proves

The canonical AGENT_REGISTRY contains six AgentSpec records with names, roles, descriptions and capability labels. This establishes declared specialist identities and metadata. It does not, by itself, establish independent reasoning, separate deployed services or autonomous tool execution.

Source: core/agent_registry.py; core/ai_core.py.

ROUTING & EXECUTION PATHS

04 / Selected-specialist dispatch

I implemented AICore.resolve_agent_name(task) as deterministic keyword routing. Planning-related keywords select Alina; execution-related terms select Kian; analysis, quality, security and finance terms select their corresponding specialists. The first matching branch wins, and Alina is the fallback. This is not a learned semantic router.

The dispatch path checks the token-optimizer cache. On a miss, it selects the specialist, constructs a prompt through agent.handle(task), chooses a model through TrustAwareModelRouter, calls the configured LLM provider, records usage and caches the response. On a cache hit, it returns cached text with a selected agent and model label, without a fresh completion.

05 / Six-specialist collection

I also implemented AICore.run_workflow(task), which iterates through Alina, Kian, Bita, Aylin, Ahmad and Amin in that order. For each specialist it invokes agent.handle(task), records usage and caches the resulting specialist text. The method returns six structured entries containing specialist name, role, response and task.

CRITICAL EXECUTION DISTINCTION | The six-specialist loop is sequential and calls specialist handlers. It does not call the LLM provider six times, and the source does not show concurrent distributed execution in this path.

06 / Provider behavior

The default provider is EchoProvider: deterministic, offline and not evidence of generative-model intelligence. An optional HTTP provider is configuration-gated; when unavailable or failing, its completion path falls back to Echo. I therefore do not describe the default deployment as a cloud-powered multi-model agent network.

07 / Concrete task example

For an illustrative task such as “validate the deployment checklist”, the routing function selects Aylin because the first applicable keyword family is quality/validation. Separately, the six-specialist collection path still returns one handler response per registered specialist. This is a code-derived example, not a measured production trace.

Sources: core/ai_core.py; core/llm_provider.py; core/model_router.py (referenced dependency).

ORCHESTRATOR CONTROL FLOW

08 / One-task coordination sequence

01 Enforce tenant budget — FinOpsAutopilot.enforce_budget runs before the retry-wrapped workflow.

02 Open reversible context — ReversibleManager begins a transaction-like local compensation sequence.

03 Record task and context — Evidence ledger append is attempted; task state is written to tenant-scoped memory.

04 Evaluate human authority — HumanAuthority checks risk; DENY raises. Approval-required metadata is stored.

05 Dispatch and collect — AICore.dispatch selects one specialist; AICore.run_workflow collects six handlers.

06 Compare shadow output — ShadowSwarmRunner performs primary/shadow dispatches and reports divergence.

07 Account and assess — Usage is recorded; ProductionQualityGate evaluates selected output.

08 Close workflow — Completion context/evidence is attempted; reversible steps are committed or rolled back.

09 / Human-control boundary

The orchestration function blocks tasks that HumanAuthority.check_authorization explicitly denies. However, when the policy reports that human approval is required, run_agent_workflow writes an approval-required memory marker and continues to dispatch. I do not claim this particular function enforces a blocking approval wait. The separate Cognitive Twin action lifecycle provides a distinct approval-bound execution mechanism.

10 / Evidence and compensation boundary

Several ledger writes and compensation registrations are enclosed in broad exception handlers. As written, failures in those blocks can be suppressed. ReversibleManager usage does not establish an atomic transaction across the ledger, cache, memory, provider and external effects. My engineering description is therefore best-effort evidence capture and local compensating steps, not guaranteed all-or-nothing rollback.

Source: app/orchestrator.py; core/human_authority.py; core/reversible_workflow.py (referenced dependency).

SHADOW COMPARISON & QUALITY

11 / What ShadowSwarmRunner actually measures

I implemented a primary/shadow comparison that calls AICore.dispatch twice. By default the shadow input appends “(shadow)” to the original task; a caller may provide another modifier. The default divergence metric is the normalized absolute difference between the character lengths of the two response strings. Consensus is divergence < 0.15.

INTERPRETATION LIMIT | This is a lightweight output-length proxy, not semantic agreement, adversarial robustness certification, factual correctness measurement or a statistically validated consensus score.

12 / Quality, budget and retry components

The orchestrator integrates ProductionQualityGate.evaluate, FinOpsAutopilot, RetryGuard and TenantMemoryVault. Their presence is directly visible at call sites. This review does not independently establish service-level availability, financial savings, measured throughput, or strict end-to-end tenant isolation across every dependency.

13 / Verification classification

Source-confirmed: six registered specialists; keyword routing; cache-aware selected dispatch; sequential six-handler collection; Echo default; orchestrator component calls; approval-required marker; two-dispatch shadow comparison; response-length divergence threshold.

Not independently verified here: a passing full test suite; production traffic; concurrency benchmarks; cross-process execution; guaranteed rollback; measured quality uplift; external tool execution by all six specialists.

14 / Engineering rationale

My design separates specialist metadata, routing logic, provider abstraction, local orchestration and the operator-facing approval lifecycle. That separation makes behavior inspectable and creates extension points. It also imposes a documentation responsibility: an extension point is not proof that the intended future capability is already shipped.

SOURCE TRACEABILITY & CLAIM BOUNDARIES

15 / Source-to-claim map

core/agent_registry.py — Six named AgentSpec roles and declared capabilities.

core/ai_core.py — Keyword resolution; cached/uncached dispatch; sequential six-handler workflow.

app/orchestrator.py — Budget, memory, evidence, authority, shadow, retry, quality and completion wiring.

core/shadow_swarm.py — Primary/shadow dispatch; length-based divergence; consensus threshold.

core/llm_provider.py — EchoProvider default; optional HTTP completion and Echo fallback.

README.md and STATUS.md — Repository-stated local-first product boundaries and shipped/not-shipped distinctions.

16 / Design strengths and current constraints

Strengths I can substantiate: readable specialist catalog, explicit routing rules, deterministic default inference, modular provider interface, traceable workflow assembly, tenant-labeled memory operations and visible hooks for budget, quality and recovery.

Constraints I explicitly disclose: keyword routing is brittle to phrasing and branch order; specialist collection is sequential; Shadow consensus is a shallow heuristic; some evidence failures are swallowed; approval-required metadata in this orchestrator does not halt dispatch; and code inspection alone does not establish production-grade reliability.

17 / Author statement

I present Aegis Agent as an evolving, local-first, human-governed agent platform with inspectable orchestration components. I do not characterize the examined implementation as a fully autonomous, distributed, parallel or independently validated production swarm. Every substantive claim in this document is bounded by the cited source paths and the explicitly stated review scope.

Repository

github.com/aminazimi42-coder/aegis-agent

Review method: read-only inspection of public repository source files. No runtime tests or performance benchmarks were executed for this document. No repository files were modified.